2 ЛАБОРАТОРНА РОБОТА № 2 КОМП'ЮТЕРНИЙ ЗІР. КЛАСИФІКАЦІЯ ТА РОЗПІЗНАВАННЯ ЗОБРАЖЕНЬ

2.1 Мета роботи

Навчитися реалізовувати програмні проєкти у сфері інженерії програмного забезпечення, пов'язані з необхідністю обробки зображень.

2.2 Короткі теоретичні відомості

Однією з важливих галузей штучного інтелекту є комп'ютерне бачення (Computer Vision, CV). **Комп'ютерне бачення** (комп'ютерний зір) – це наука про комп'ютери та програмні системи, які можуть розпізнавати та розуміти зображення та сцени. Комп'ютерне бачення дозволяє комп'ютеру імітувати людський зір і приймати рішення на основі побачених сцен [9], [10]. Комп'ютерне бачення також складається з різних аспектів, таких як розпізнавання зображень (image recognition), виявлення об'єктів (object detection), створення зображень (image generation), збільшення якості зображень за допомогою методів надроздільної здатності (image super-resolution) тощо [9].

2.2.1 Розпізнавання зображень

Розпізнавання зображень стосується аналізу пікселів і шаблонів зображення з метою розпізнавання зображення як окремого об'єкту [10]. Поширеним прикладом розпізнавання зображень є оптичне розпізнавання символів. Сканер може ідентифікувати символи на зображенні, щоб перетворити текст із зображення в текстовий файл. За допомогою того самого процесу можна розпізнавати, наприклад, текст номерного знака на зображенні [10].

Для розроблення програмного забезпечення для розпізнавання зображень необхідно синтезувати розпізнавальну модель на основі наявних даних. Процес **створення моделі розпізнавання зображень** не відрізняється від процесу моделювання у машинному навчанні [10], основні етапи цього процесу наведено нижче.

Етап 1. Витягання піксельних функцій із зображення (Extract pixel features from an image).

На цьому етапі виконується оцифровка зображення з реального світу, внаслідок чого із зображення витягується велика кількість характеристик (ознак). Внаслідок цього отримується цифрове зображення, яке складається з пікселів (рис.2.1). Кожен піксель представляється числом або набором чисел (діапазон цих чисел називається глибиною кольору або бітовою глибиною). Глибина кольору вказує на максимальну кількість потенційних кольорів, які можна використовувати в зображенні. У 8-бітному сірому зображенні кожен піксель має одне значення в діапазоні від 0 до 255. Більшість зображень використовують 24-бітний колір або вище. Зображення з роздільною здатністю 1024 × 768 являє собою сітку з 1024 стовпцями та 768 рядками, яка, отже, містить 1024 × 768 = 0,78 мегапікселя [10].

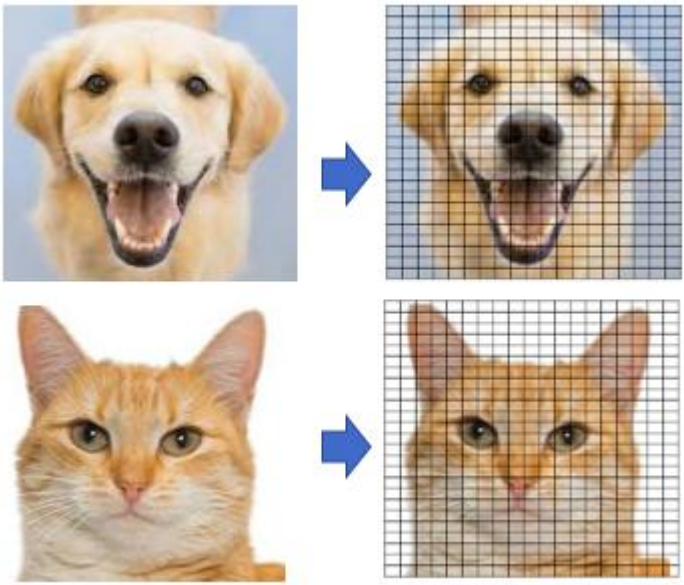


Рисунок 2.1 – Витягання піксельних функцій із зображення [10]

Етап 2. Підготовка множини зображень з мітками для навчання моделі (формування навчальної вибірки).

На цьому етапі створюється велика кількість зображень, що відносяться до різних категорій (міток, класів), наприклад, «собака»,

«кіт», «риба», «птах» та ін. (рис. 2.2). Чим більше зображень буде використано для кожної категорії, тим краще можна навчити модель визначати відповідну категорію (клас) зображення. На цьому етапі відомою є категорія, до якої належить кожне зображення [10]. Ці дані (зображення та категорії, до яких вони відносяться) будуть використовуватися на наступному етапі для навчання моделі.

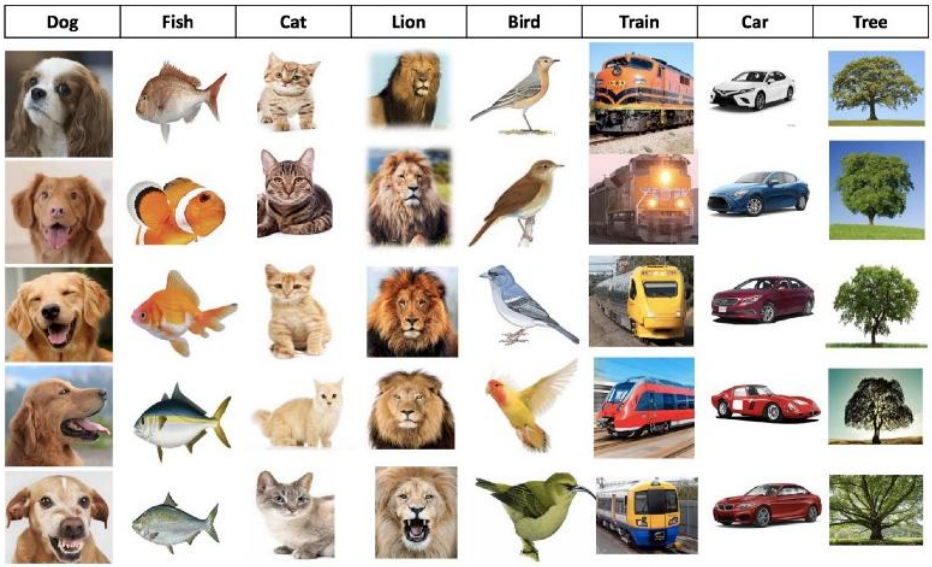


Рисунок 2.2 – Формування навчальної вибірки [10]

Етап 3. Моделювання (синтез розпізнавальної моделі).

На цьому етапі відбувається синтез моделі (підбір значень її параметрів) шляхом її навчання до класифікації наявних зображень (зображень, поданих у навчальній вибірці). На рис.2.3 показано, як модель навчається за допомогою попередньо позначених зображень (зображень, для яких відомими є категорії, класи). Зображення у витягнутих формах надходять на вхідну сторону моделі, а мітки (категорії, класи) – на вихідну. Мета тут полягає в тому, щоб навчити моделі (як правило, нейронні мережі) таким чином, щоб зображення зі своїми характеристиками, що надходять із вхідних даних, відповідало мітці праворуч [10].

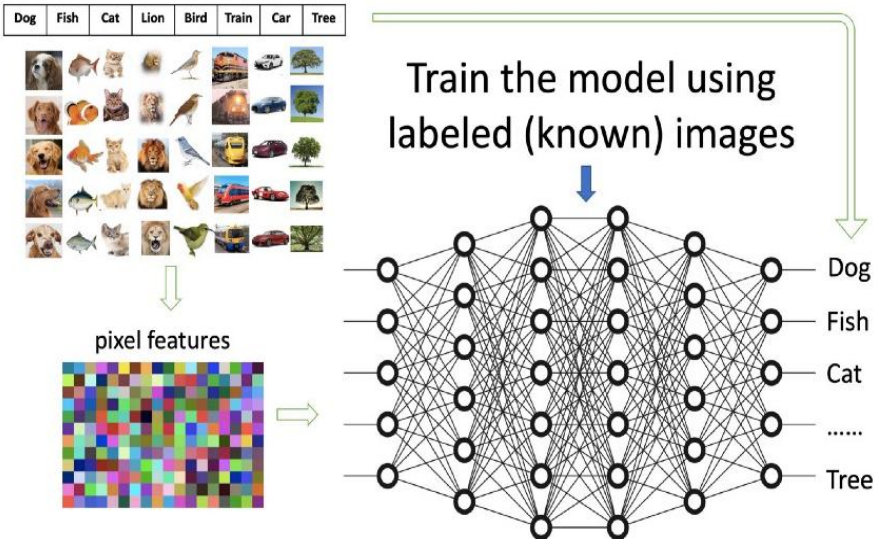


Рисунок 2.3 – Навчання розпізнавальної моделі [10]

Етап 4. Використання синтезованої моделі для розпізнавання нових зображень.

Коли модель навчена, її можна використовувати для розпізнавання невідомого зображення. На рис. 2.4 показано, що нове зображення розпізнається як зображення собаки [10].

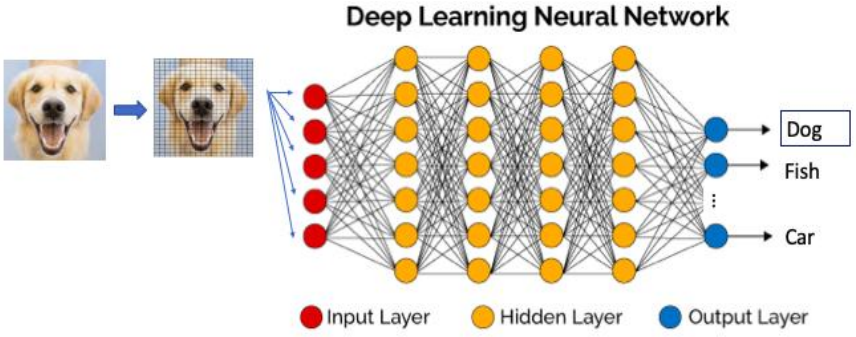


Рисунок 2.4 – Використання синтезованої моделі для розпізнавання нових зображень [10]

Як базис для створення розпізнавальних моделей ефективно застосовуються **згорткові нейронні мережі** (Convolution Neural Networks, CNN), які широко застосовуються для класифікації зображень, виявлення об'єктів або розпізнавання зображень (рис. 2.5) [10].

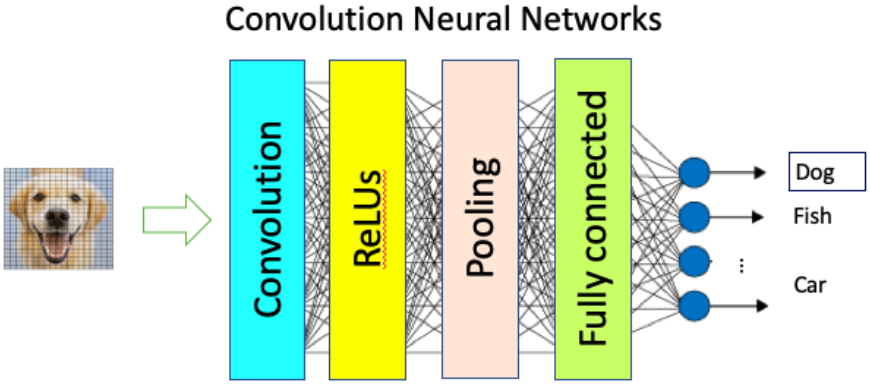


Рисунок 2.5 – Застосування згорткових нейронних мереж розпізнавання зображень [10]

У мережах CNN існує чотири основні типи шарів: згортка (convolution), ReLU, об'єднання (pooling) та повнозв'язані шари (fully nnected), як показано на рис. 2.5. Для пояснення роботи цих шарів розглянемо приклад застосування згорткових нейронних мереж для розпізнавання зображень рукописних символів (чисел від 0 до 9) у вигляді зображень у тонах сірого вільної форми [10].

Шар згортки (convolution layer). Першим кроком, який виконують мережі CNN, є створення багатьох маленьких елементів, які називаються ознаками (features), як-от квадрати розміром 2x2. Кожна ознака характеризує певну форму вихідного зображення. При скануванні зображення за допомогою кожної ознаки розраховується відповідна оцінка частини зображення. Якщо збіг є ідеальним, у цієї частини зображення (наприклад, квадрату розміром 2x2), їй присвоюється у відповідність високий бал. Якщо збіг є низьким (або взагалі відсутній), оцінка відповідності також є низькою або нульовою. Цей процес отримання балів (scores) називається фільтрацією (filtering) [10].

На рис. 2.6 показано три функції. Кожна функція створює відфільтроване зображення з високими та низькими оцінками під час ска-

нування вихідного зображення. Наприклад, червоне поле знайшло чотири області на оригінальному зображенні, які ідеально збігаються з об’єктом, тому для цих чотирьох областей оцінки високі. Рожеві прямокутники – це області, які певною мірою збігаються. Акт перевірки кожного можливого збігу шляхом сканування вихідного зображення називається згорткою. Відфільтровані зображення складаються разом, щоб стати шаром згортки [10].

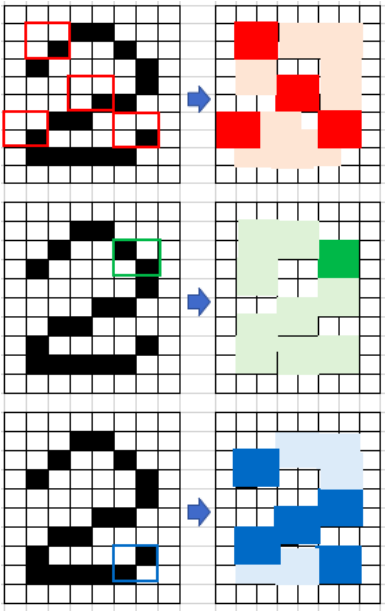


Рисунок 2.6 – Перетворення зображень за допомогою шару згортки [10]

Шар ReLU (Rectified Linear Unit) виправляє будь-яке від’ємне значення до нуля, щоб гарантувати, що нейронна мережа буде працювати коректно. Математично функція ReLU являє собою виправлену лінійну (кусково-лінійну) функцію активації, яка повертає вхідне значення без змін, якщо воно є додатнім, в іншому випадку повертається нульове значення [10].

Шар Max Pooling (максимального об'єднання) відповідає за зменшення розміру зображення. На рис. 2.7 вікно 2х2 сканує кожне з відфільтрованих зображень і призначає максимальне значення цього вік-

на 2x2 рамці 1x1 у новому зображенні. Як показано на рисунку, максимальне значення в першому вікні 2x2 є високим балом (позначеним червоним кольором), тому високий бал призначається полю 1x1 нового зображення. Поле 2x2 переміщується до другого вікна, де є високий бал (червоний) і низький бал (рожевий), тому високий бал призначається наступному полю 1x1. Наступному полю призначається світло червоний колір. Після об'єднання створюється новий стек менших відфільтрованих зображень [10].

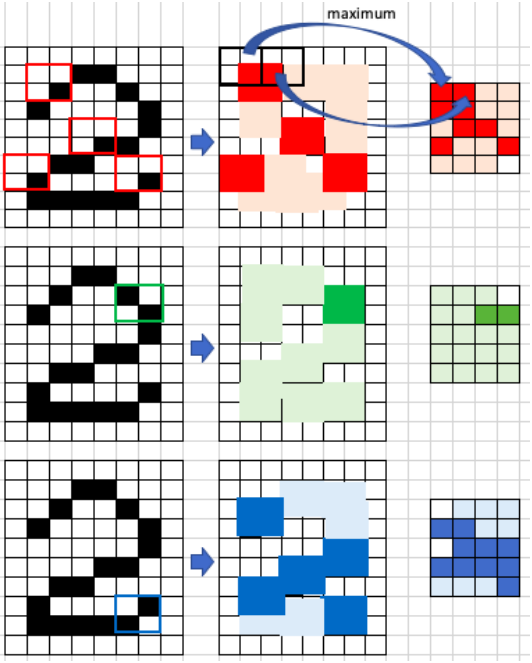


Рисунок 2.7 – Перетворення зображень за допомогою шару Max Pooling [10]

Повноз'вязний шар (фінальний шар). На цьому шарі відбувається розділення менш відфільтрованих зображень та складання їх в один список, як показано на рис.2.8. Кожне значення в єдиному списку передбачає ймовірність для кожного з кінцевих значень 1, 2,… і 0. Ця частина є такою ж, як вихідний шар у звичайних (багатошарових) нейронних мережах. У нашому прикладі цифра «2» отримує найвищу

загальну оцінку з усіх вузлів єдиного списку. Тому CNN розпізнає оригінальне зображення рукописного тексту як «2» [10].

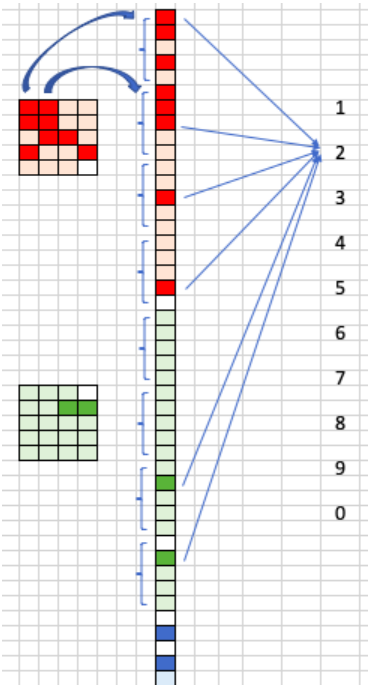


Рисунок 2.8 – Робота останнього, повноз'вязного шару при розпізнаванні зображень [10]

Відзначимо, що багатошарові нейронні мережі складають вихідне зображення у список і перетворюють його на вхідний шар. Інформація між сусідніми пікселями при цьому може не зберігатися. На відміну від цього, CNN створює шар згортки, який зберігає інформацію про сусідні пікселі, тому розпізнавання за допомогою CNN є більш ефективним [10].

Окрім нейромоделей типу CNN для розпізнавання зображень також ефективно можуть використовуватися попередньо навчені глибокі моделі (deep learning models), які є у відкритому доступі, зокрема, Xception, VGG16, VGG19, ResNet50, InceptionV3, InceptionResNetV2, MobileNet, DenseNet, NASNet і MobileNetV2 [11].

Ці моделі можна використовувати для розпізнавання та прогнозування, виділення ознак і точного налаштування.

Також у відкритому доступі є досить велика базу даних зображень ImageNet, яку можна завантажити для дослідницьких цілей при розв'язанні різних завдань обробки зображень [12].

2.2.2 Виявлення об'єктів на графічних зображеннях

Виявлення об'єктів (object detection) означає здатність комп'ютерних і програмних систем знаходити об'єкти на зображенні/сцені та ідентифікувати кожен об'єкт. Виявлення об'єктів широко використовується для виявлення облич на зображеннях, виявлення транспортних засобів, підрахунку пішоходів, обробки вебзображень, у системах безпеки та програмних засобів для автомобілів без водія. Виявлення об'єктів можна використовувати багатьма способами в багатьох галузях практики [9].

Ранні реалізації виявлення об'єктів передбачали використання класичних алгоритмів, подібних до тих, що підтримуються в OpenCV, популярній бібліотеці комп'ютерного зору. Однак ці класичні алгоритми не могли досягти достатньої продуктивності для роботи в інших умовах. У наш час для виявлення об'єктів використовуються більш високоточні алгоритми і моделі, зокрема як R-CNN, Fast-RCNN, Faster-RCNN, RetinaNet, SSD і YOLO. Використання цих методів і алгоритмів, заснованих на нейронних мережах та глибокому навчанні, яке також базується на машинному навчанні, потребує великої кількості математичних розрахунків [9].

Тому розроблено багато програмних бібліотек, що дозволяють розробникам програмного забезпечення легко інтегрувати сучасні технології комп'ютерного зору у розроблювані програми, використовуючи лише кілька рядків коду. Роботу з такими бібліотеками описано, наприклад, у джерелах [9], [13]–[19].

2.3 Приклад розробки програмного забезпечення для обробки зображень

Як приклади, будемо використовувати програмні коди, подані у джерелах [9], [13]–[19].

Програмне забезпечення буде розроблюватися з використанням бібліотеки tensorflow та мови Python.

2.3.1 Розробка програмного забезпечення для розпізнавання графічних об'єктів

Розглянемо навчання простої згорткової нейронної мережі (CNN) для розпізнавання (класифікації) зображень з набору CIFAR [13]. Спочатку імпортуємо необхідні для роботи програмного забезпечення бібліотеки:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

Виконаємо завантаження набору даних CIFAR10, який містить 60 000 кольорових зображень, кожне з яких відноситься до одного з 10 класів, по 6 000 зображень у кожному класі. Набір даних поділено на 50 000 навчальних зображень та 10 000 тестових зображень. Класи взаємовиключні і з-поміж них немає перетину.

```
(train_images, train_labels), (test_images,
test_labels) = datasets.cifar10.load_data()
```

Збережемо навчальні та тестові даних у файлах train.npz та test.npz для можливості їх подальшого завантаження

```
np.savez("train.npz", train_images=train_images,
train_labels=train_labels)
np.savez("test.npz", test_images=test_images,
test_labels=test_labels)
```

При необхідності повторного використання набору CIFAR10 завантажити збережені раніше дані можна таким чином:

```
data = np.load("train.npz")
train_images = data['train_images']
train_labels = data['train_labels']

data = np.load("test.npz")
test_images = data['test_images']
test_labels = data['test_labels']
```

Нормалізація піксельних значень у межах від 0 до 1:

```
train_images, test_images = train_images / 255.0,
test_images / 255.0
```

Щоб переконатися, що набір даних виглядає правильно та є коректним, побудуємо перші 25 зображень із навчального набору та відобразимо назву класу під кожним зображенням:

```
class_names = ['airplane', 'automobile', 'bird', 'cat',
'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```

В результаті отримаємо рисунок, на якому відображено перші 25 зображень з завантаженого набору (рис. 2.9).

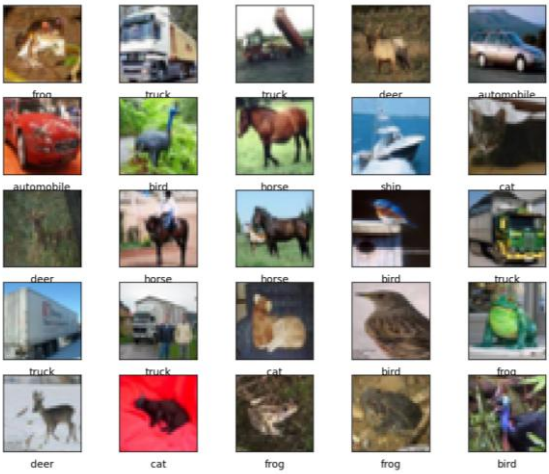


Рисунок 2.9 – Фрагмент вхідних даних

На наступному етапі виконаємо створення згорткової нейронної мережі.

Перші шість рядків коду визначають базу згортки за допомогою загального шаблону: стек шарів Conv2D і MaxPooling2D. Як вхідні дані CNN приймає тензори (набори даних) форми (висота_зображення, ширина_зображення, канали_кольору). У цьому прикладі модель CNN буде обробляти вхідні дані форми input_shape=(32, 32, 3), що є форматом зображень CIFAR (32 – висота, 32 – ширина, 3 – кількість кольорових каналів, у даному випадку три, оскільки використовується кольорова модель R,G,B).

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

Виведемо архітектуру моделі на екран:

```
model.summary()
```

Як результат, отримаємо таку архітектуру моделі model:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 30, 30, 32)        896

 max_pooling2d (MaxPooling2D) (None, 15, 15, 32)       0

 conv2d_1 (Conv2D)           (None, 13, 13, 64)        18496

 max_pooling2d_1 (MaxPooling  (None, 6, 6, 64)         0
 2D)

 conv2d_2 (Conv2D)           (None, 4, 4, 64)          36928
=================================================================
Total params: 56,320
Trainable params: 56,320
Non-trainable params: 0
```

Як видно, результатом кожного шару Conv2D і MaxPooling2D є тривимірний тензор форми (висота, ширина, канали). Розміри ширини

та висоти мають тенденцію до зменшення в міру заглиблення в мережу. Кількість вихідних каналів для кожного рівня Conv2D контролюється першим аргументом (наприклад, 32 або 64). Як правило, коли ширина та висота зменшуються, ви можете дозволити собі (з точки зору обчислень) додати більше вихідних каналів у кожен шар Conv2D.

Щоб завершити модель, необхідно передати останній вихідний тензор із згорткової бази (форми (4, 4, 64)) в один або декілька Dense-шарів (звичайних шарів нейронів, аналогічних тим, які використовуються у багатошарових нейромоделях) для виконання класифікації. Dense-шари приймають вектори як вхідні дані (які є одновимірними), тоді як поточний вихід є 3D-тензором. Тому спочатку необхідно звести (або розгорнути) 3D-вихід до одновимірного (за допомогою шару Flatten), а потім додати один або декілька Dense-шарів зверху. Набір даних CIFAR має 10 вихідних класів, тому на останньому шарі необхідно використовувати 10 виходів (10 нейронів):

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

Використовуючи команду `model.summary()` , виведемо архітектуру моделі на екран:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 30, 30, 32)        896

 max_pooling2d (MaxPooling2D) (None, 15, 15, 32)       0

 conv2d_1 (Conv2D)           (None, 13, 13, 64)        18496

 max_pooling2d_1 (MaxPooling  (None, 6, 6, 64)         0
 2D)

 conv2d_2 (Conv2D)           (None, 4, 4, 64)          36928

 flatten (Flatten)           (None, 1024)              0

 dense (Dense)               (None, 64)                65600

 dense_1 (Dense)             (None, 10)                650
=================================================================
Total params: 122,570
Trainable params: 122,570
Non-trainable params: 0
```

Як видно, останній вихідний тензор згортки (форми (4, 4, 64)) було зведено у вектори форми (1024) перед проходженням через два Dense-шари.

Після цього виконується компіляція на навчання згорткової нейронної мережі:

```
model.compile(optimizer='adam',
loss=tf.keras.losses.SparseCategoricalCrossentropy(from
_logits=True), metrics=['accuracy'])

history = model.fit(train_images, train_labels,
epochs=10, steps_per_epoch=100, valida-
tion_data=(test_images, test_labels))
```

На останньому етапі виконується оцінювання моделі та відображення результатів її роботи

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label =
'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')

plt.show()

test_loss, test_acc = model.evaluate(test_images,
test_labels, verbose=2)

print("Точність моделі на тестових даних: ", test_acc)
```

Як результат, відображається точність моделі на тестових даних. Враховуючи стохастичний характер синтезу нейромоделі, точність при кожному запуску програми буде різною.

Також відображається графік залежності точності (Accuracy) розпізнавання зображень за допомогою моделі від номеру ітерації (епохи, Epoch) для навчальних та перевірочних даних (рис. 2.10).

Після синтезу моделі, її можна використовувати для класифікації зображень, що не входили у навчальну вибірку.

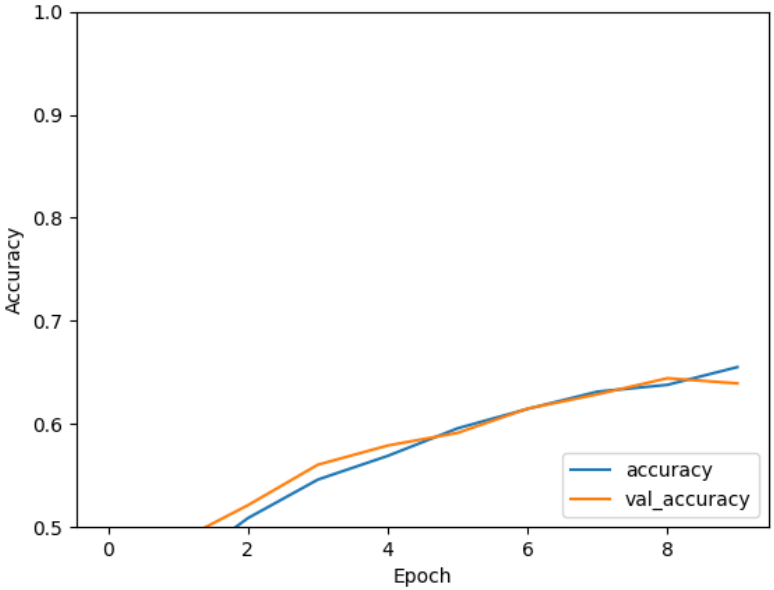


Рисунок 2.10 – Графік залежності точності розпізнавання зображень за допомогою синтезованої моделі від номеру ітерації

Також рекомендовано ознайомитися з іншим прикладом використання згорткової нейронної мережі для розпізнавання зображень, поданим у джерелі [14].

2.3.2 Розробка програмного забезпечення для пошуку об’єктів на графічних зображеннях

У наш час розроблено багато моделей, які програмно реалізовані та є загально доступними, для пошуку об’єктів (object detection) на графічних зображеннях. Задача синтезу таких моделей і доволі складною та вимагає багато часу та обчислювальних ресурсів комп’ютера, тому будемо використовувати вже існуючі програмні бібліотеки та моделі пошуку об’єктів.

Розробимо програмне забезпечення для пошуку об’єктів на графічних зображеннях за допомогою бібліотеки opencv-python. Отже для роботи програми необхідно встановити деякі бібліотеки (якщо цього не було зроблено раніше):

```
pip install opencv-python
pip install matplotlib
```

Завантажимо програмні модулі, які необхідні для роботи програми для пошуку облич на графічних зображеннях, а також очей на знайдених зображеннях:

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
```

Відзначимо, що для роботи програми необхідно завантажити попередньо підготовлені класифікатори OpenCV, які містять інформацію для розпізнавання відповідних об'єктів (обличчя, очі та ін.) на зображеннях. У даному випадку заздалегідь підготовлені файли haarcascade_frontalface_default.xml та haarcascade_eye.xml, які знаходяться у відкритому доступі та містять інформацію для детекції облич на зображеннях, а також очей на обличчях, відповідно [18]:

```
face_cascade =
cv2.CascadeClassifier('haarcascade_frontalface_default.
xml')
eye_cascade =
cv2.CascadeClassifier('haarcascade_eye.xml')
```

Завантажимо зображення з файлу photo.jpg та виконаємо його перетворення тони сірого:

```
img = cv2.imread('photo.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Знаходимо обличчя на зображенні за допомогою класифікатора face_cascade:

```
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
```

Знаходимо очі на попередньо знайдених обличчях:

```
for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = img[y:y+h, x:x+w]
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex,ey,ew,eh) in eyes:
```

```
cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(0,255,0)
,2)
```

Відображаємо результат:

```
cv2.imshow('img',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Після запуску програми виводиться зображення з файлу photo.jpg з позначеними на ньому обличчями та очима людей.

Зауважимо, що якість пошуку об'єктів на зображеннях залежить не тільки від самих зображень, а і від якості попередньо синтезованих детекторів.

Також програмне забезпечення для пошуку об'єктів на графічних зображеннях може бути розроблено за допомогою бібліотеки ImageAI [9], написаної на мові Python з використанням засобів бібліотеки tensorflow та деяких інших бібліотек Python.

Отже для роботи з бібліотекою ImageAI необхідно встановити деякі інші бібліотеки (якщо цього не було зроблено раніше):

```
pip install tensorflow
pip install TorchVision keras numpy pillow scipy h5py
matplotlib opencv-python keras-resnet
```

Після цього встановимо бібліотеку ImageAI:

```
pip install imageai --upgrade
```

Виконаємо завантаження файлу з вже синтезованою моделлю RetinaNet для пошуку об'єктів на графічних зображеннях [19]:

https://github.com/OlafenwaMoses/ImageAI/releases/download/essentials-v5/resnet50_coco_best_v2.1.0.h5/

Створимо файл LR2_imageDetection.py з таким програмним кодом:

```
from imageai.Detection import ObjectDetection

import os

execution_path = os.getcwd()

detector = ObjectDetection()
```

```
detector.setModelTypeAsRetinaNet()
detector.setModelPath( os.path.join(execution_path ,
"resnet50_coco_best_v2.1.0.h5"))
detector.loadModel()
detections =
detector.detectObjectsFromImage(input_image=os.path.joi
n(execution_path , "image.jpg"),
output_image_path=os.path.join(execution_path ,
"imagenew.jpg"))

for eachObject in detections:
    print(eachObject["name"] , " : " , eachObject["percentage_probability"] )
```

Після запуску код у консолі друкуються результати роботи програми: назва виділеного об'єкту (name, наприклад, person, bus, car) та відсоток ймовірності (percentage_probability), що виділений об'єкт відноситься до конкретного типу. Коли результат буде надруковано на консолі (програма виконає обробку заданого файлу), можна перейти до папки, у якій знаходиться програмний файл LR2_imageDetection.py, де буде знаходитися нове зображення з виділеними на ньому об'єктами.

2.4 Завдання на лабораторну роботу

2.4.1 Ознайомитися з основними теоретичними відомостями та рекомендованою літературою за темою роботи.

2.4.2 Обрати вхідні дані, подані у графічних зображень, для подальшої їх обробки та аналізу.

Як вхідні дані можна обрати, наприклад, такі:

– дані з відомих репозиторіїв. Наприклад, з репозиторію TensorFlow [20]:

https://www.tensorflow.org/datasets ;

– дані з предметної області, що є добре відомою для студента;

– спеціально згенеровані вхідні дані;

– будь-які інші дані, які подані у вигляді графічних зображень.

2.4.3 Розробити програмне забезпечення для вирішення однієї з задач комп'ютерного зору (розпізнавання зображень, пошуку об'єктів на графічних зображеннях та ін.).

2.4.3.1 Для створення програмного забезпечення обрати мову програмування та інші засоби розробки (програмні бібліотеки та ін.). Студент не обмежується у виборі мови програмування та інших засоби розробки програмного забезпечення.

2.4.3.2 Проаналізувати обрані вхідні дані та предметну область, визначити вимоги до програмного забезпечення (формати подання вхідних даних, необхідність попередньої обробки даних, необхідність поділу множини даних на навчальну та тестову вибірки даних, використовувані математичні моделі для обробки зображень, необхідність подання результатів у вигляді графіків та ін.).

2.4.3.3 Здійснити проєктування архітектури програмного забезпечення для обробки зображень.

2.4.3.4 Виконати конструювання програмного забезпечення.

2.4.3.5 Виконати тестування розробленого програмного забезпечення. Подати результати тестування розробленого програмного забезпечення для обробки зображень у зручному для подальшого аналізу вигляді.

2.4.4. Зробити висновки за роботою.

2.4.5. Оформити звіт з роботи.

2.4.6. Відповісти на контрольні питання.

2.5 Зміст звіту

– назва і мета роботи;

– фрагмент вхідних даних та їх короткий опис;

– текст розробленого програмного забезпечення;

– результати аналізу вхідних даних за допомогою розробленого програмного забезпечення;

– висновки, що відображують результати виконання лабораторної роботи.

2.6 Контрольні запитання

2.6.1 Поясніть поняття «Комп’ютерне бачення».

2.6.2 Які задачі вирішує комп’ютерне бачення?

2.6.3 У чому полягає суть розпізнавання зображень?

2.6.4 Проаналізуйте етапи створення моделі розпізнавання зображень.

2.6.5 Яким чином відбувається формування навчальної вибірки для синтезу моделі для розпізнавання зображень?

2.6.6 У чому полягає етап синтезу розпізнавальної моделі для обробки зображень?

2.6.7 Як використовуються синтезовані моделі для розпізнавання нових зображень?

2.6.8 Які типи шарів нейронів використовуються у згорткових нейронних мережах?

2.6.9 Для чого призначений шар згортки у згорткових нейромоделях?

2.6.10 Яким чином будується останній шар згорткової нейронної мережі для розпізнавання зображень? Скільки нейронів містить такий шар?

2.6.11 Чому для розпізнавання зображень більш доцільно використовувати згорткові нейронні мережі, ніж багатошарові персептрони?

2.6.12 Які типи попередньо навчених глибоких моделей можуть використовуватися для розпізнавання зображень?

2.6.13 Що таке виявлення (детекція) об’єктів на зображеннях?

2.6.14 У яких практичних застосуваннях використовується виявлення об’єктів на зображеннях?

2.6.15 Які моделі доцільно використовувати для виявлення об’єктів на зображеннях?